

# Pearls AQI Predictor

*End-to-End MLOps System for 3-Day Air Quality Forecasting*

Karachi, Pakistan

---

**Author:** Shayan Khan Afridi

---

Project Repository:

[github.com/ShayanKhanAfridi/pearls-aqi-predictor](https://github.com/ShayanKhanAfridi/pearls-aqi-predictor)

Live Dashboard:

[pearls-aqi-predictor-by-shayan.streamlit.app](https://pearls-aqi-predictor-by-shayan.streamlit.app)

# 1. Executive Summary

Pearls AQI Predictor is a fully automated, production-grade Machine Learning Operations (MLOps) system built to forecast the Air Quality Index (AQI) for Karachi, Pakistan over the next three days (72 hours). The system collects real-time environmental data from the Open-Meteo API, engineers a rich set of 42 features, trains and compares five machine learning algorithms, and delivers predictions through a modern Streamlit web dashboard — all without any manual intervention.

The architecture follows a serverless, cloud-first design: Hopsworks Feature Store serves as the central feature repository, GitHub Actions automates hourly data ingestion and daily model retraining, and the best-performing model is automatically registered in the Hopsworks Model Registry. The dashboard performs live batch inference on every page load, ensuring users always see the most current 72-hour AQI forecast.

## Key Technical Highlights:

- Multi-output horizon forecasting: 3 targets (Day 1, Day 2, Day 3) predicted simultaneously
- 42 engineered features: lags, rolling statistics, cyclical encodings, wind vectors, regime features
- 5 candidate models evaluated: Ridge, Random Forest, XGBoost, LightGBM, Gradient Boosting
- Automated selection via Weighted RMSE ( $\text{Day1} \times 0.20 + \text{Day2} \times 0.35 + \text{Day3} \times 0.45$ )
- Serverless architecture: Hopsworks (Feature Store + Model Registry) + GitHub Actions CI/CD
- Gaussian-weighted target construction for Day 2 and Day 3 to reduce prediction noise

## 2. Project Overview

### 2.1 Motivation and Problem Statement

Air quality in Karachi is a serious and persistent public health concern. The city regularly experiences elevated particulate matter (PM2.5 and PM10) concentrations driven by industrial activity, vehicular emissions, sea breezes, and seasonal monsoon patterns. Unlike a standard next-hour forecast, residents and city planners need multi-day outlooks so they can plan outdoor activities, issue public health advisories, and take preventive action.

Existing commercial AQI services either lack Karachi-specific granularity or do not expose raw forecasts in a usable form. The goal of this project was to build a self-sustaining MLOps pipeline that solves the forecasting problem end-to-end: from raw API data to a live dashboard, with zero daily human involvement.

### 2.2 Project Objectives

1. Collect real-time and historical AQI and weather data from Open-Meteo APIs.
2. Engineer a comprehensive feature set capturing temporal patterns, pollutant history, and atmospheric conditions.
3. Design a multi-output forecasting model that simultaneously predicts AQI for Day 1, Day 2, and Day 3.
4. Automate the full ML lifecycle — feature ingestion, model retraining, and model registration — via GitHub Actions.
5. Build an interactive Streamlit dashboard for live inference and historical analysis.
6. Implement explainability tooling (SHAP) to understand model decisions.

### 2.3 System Architecture Overview

The system is organized into five functional components that form a continuous loop:

| Component           | Description                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------|
| Backfill Pipeline   | One-time ingestion of 3 years of historical AQI and weather data into the Feature Store       |
| Feature Pipeline    | Hourly live data collection, feature engineering, and Feature Store update via GitHub Actions |
| Training Pipeline   | Daily model training across 5 algorithms, selection by Weighted RMSE, and registry upload     |
| Hopsworks Platform  | Serverless Feature Store and Model Registry hosted on eu-west.cloud.hopsworks.ai              |
| Streamlit Dashboard | 5-page interactive web app for inference, historical analysis, and health advisories          |
| GitHub Actions      | CI/CD automation: hourly feature pipeline and daily training pipeline on schedule             |

## 3. Data Collection and Sources

### 3.1 Primary Data Sources

All data is sourced from Open-Meteo — a free, open-source weather and air quality API that provides global coverage at hourly resolution without requiring API keys for basic access. Two endpoints are used:

| API Endpoint                                     | Variables Collected                                                                             |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Air Quality API (air-quality-api.open-meteo.com) | US AQI, PM2.5, PM10, CO, NO <sub>2</sub> , SO <sub>2</sub> , O <sub>3</sub>                     |
| Weather Forecast API (api.open-meteo.com)        | Temperature, Humidity, Surface Pressure, Wind Speed, Wind Direction, Cloud Cover, Precipitation |

For the 72-hour forecast horizon, the weather API is also queried for future wind speed and precipitation at +24h, +48h, and +72h — these are used as predictive features in the model.

### 3.2 Geographic Configuration

City: Karachi, Pakistan  
Latitude: 24.9056° N  
Longitude: 67.0822° E  
Timezone: Asia/Karachi (UTC+5)  
Data Granularity: Hourly  
Historical Coverage: ~3 years (backfill)

### 3.3 Historical Backfill Pipeline

The backfill pipeline (backfill\_pipeline.py) was executed once to populate the Feature Store with historical data. This was run on Google Colab rather than locally because processing and uploading three years of data involves heavy computation that would timeout on standard hardware. The pipeline:

- Fetches historical hourly AQI and weather from the Open-Meteo Archive API
- Reconstructs all engineered features for each timestamp (same logic as live pipeline)
- Stores the result to the Hopsworks Feature Group aqi\_features (version 1)

The backfill phase created the foundational training dataset that all subsequent model training builds upon.

## 4. Feature Engineering

### 4.1 Overview

One of the most critical decisions in the project was determining what features to engineer. The raw API response gives only 14 variables. After iterative experimentation, the final feature set expanded to 42 features organized into six categories:

### 4.2 Lag Features (7 features)

Lag features capture recent AQI history, which is highly predictive of future AQI given the autocorrelated nature of air quality. The following lags were selected based on physical significance:

| Feature    | Description                                               |
|------------|-----------------------------------------------------------|
| aqi_lag1   | AQI 1 hour ago — captures immediate momentum              |
| aqi_lag3   | AQI 3 hours ago — short-term trend                        |
| aqi_lag6   | AQI 6 hours ago — half-day cycle anchor                   |
| aqi_lag12  | AQI 12 hours ago — diurnal half-period                    |
| aqi_lag24  | AQI 24 hours ago — same time yesterday                    |
| aqi_lag48  | AQI 48 hours ago — 2-day memory                           |
| aqi_lag168 | AQI 7 days ago — same time last week (weekly seasonality) |

### 4.3 Rolling Statistics (8 features)

Rolling windows capture the distributional state of AQI over recent periods, which is important for understanding whether the city is in a "high pollution regime" or a clean spell:

- Rolling means over 6h, 24h, 72h, 168h windows
- Rolling standard deviations over 24h and 72h (volatility measures)
- Rolling maximum and minimum over 24h (regime range)

### 4.4 Cyclical Time Encoding (8 features)

Raw time integers (hour=0 to 23) create an artificial discontinuity at midnight. Sine/Cosine encoding ensures the model sees time as a continuous cycle:

```
hour_sin = sin(2π × hour / 24)    hour_cos = cos(2π × hour / 24)
month_sin = sin(2π × month / 12)   month_cos = cos(2π × month / 12)
dow_sin = sin(2π × day_of_week / 7) dow_cos = cos(2π × day_of_week / 7)
doy_sin = sin(2π × day_of_year / 365) doy_cos = cos(2π × day_of_year / 365)
```

### 4.5 Wind Vector Decomposition (2 features)

Wind speed and direction are combined into Cartesian components to make them numerically meaningful for tree-based models (which cannot natively understand angular variables):

```
wind_x = wind_speed × cos(wind_direction_radians)
wind_y = wind_speed × sin(wind_direction_radians)
```

## 4.6 Regime and Residual Features (3 features — v6 addition)

These features were added in pipeline version 6 after observing that the model struggled with sudden pollution spikes and regime shifts. They encode whether the current AQI is anomalous relative to its own recent baseline:

| Feature           | Formula and Meaning                                     |
|-------------------|---------------------------------------------------------|
| aqi_residual_168h | lag1 - rolling_mean_168h: deviation from 7-day baseline |
| aqi_residual_72h  | lag1 - rolling_mean_72h: deviation from 3-day baseline  |
| aqi_regime_range  | rolling_max_24h - rolling_min_24h: 24h pollution spread |

## 4.7 Interaction Features (4 features)

Cross-product features help tree-based models capture joint effects without relying on deep splits:

- wind\_x\_pm25: Wind east-west component × PM2.5 (dispersion efficiency)
- humidity\_x\_pm25: Humidity × PM2.5 (hygroscopic growth of particles)
- temp\_x\_aqi: Temperature × AQI (thermal inversion proxy)
- pollution\_index:  $0.5 \times \text{PM}_{2.5} + 0.3 \times \text{PM}_{10} + 0.2 \times \text{NO}_2$  (composite load)

## 4.8 Seasonal and Calendar Flags

- is\_weekend: Binary flag for Saturday/Sunday (reduced industrial activity)
- is\_winter: Flag for November–February (temperature inversions trap pollutants)
- is\_monsoon: Flag for July–September (rain washes particulates from air)

## 5. Training Pipeline and Model Selection

### 5.1 Multi-Output Horizon Design

The most consequential architectural decision in this project was choosing between a recursive (chained) forecasting approach and a direct multi-output approach. This decision took the most experimentation and iteration to resolve.

Approach Considered #1 — Recursive Forecasting:

Train one model to predict AQI at  $t+1$ . Feed that prediction back as input to forecast  $t+2$ ,  $t+3$ , ...,  $t+72$ .

Problem: errors compound exponentially over 72 steps. A 5-AQI error at  $t+1$  becomes 15+ by  $t+72$ .

Approach Considered #2 — Single Multi-Output Model (CHOSEN):

Train one model that simultaneously outputs three targets: Day1\_AQI, Day2\_AQI, Day3\_AQI.

Each output is a Gaussian-weighted average of AQI values over the respective 24-hour window.

Benefit: no error propagation, naturally captures cross-horizon dependencies.

### 5.2 Target Variable Construction

Targets are not single-point values but Gaussian-weighted averages over each 24-hour window. This was another key insight — predicting the mean AQI of an entire day is a more stable and meaningful forecasting task than predicting a single hour:

- Day 1 target: simple mean of AQI values over hours +1 to +24
- Day 2 target: Gaussian-weighted mean over hours +25 to +48 ( $\sigma=8.0$ )
- Day 3 target: Gaussian-weighted mean over hours +49 to +72 ( $\sigma=8.0$ )

The Gaussian weighting for Day 2 and Day 3 down-weights outlier hours at the edges of each window, making the targets smoother and easier to regress against.

### 5.3 Candidate Models

| Algorithm         | Configuration Summary                                                                                         |
|-------------------|---------------------------------------------------------------------------------------------------------------|
| Ridge Regression  | StandardScaler + MultiOutputRegressor, $\alpha=10.0$ — linear baseline                                        |
| Random Forest     | 400 estimators, $\text{max\_depth}=6$ , $\text{min\_samples\_leaf}=30$ , $\text{max\_features}=0.4$           |
| XGBoost           | 600 estimators, $\text{max\_depth}=4$ , $\text{lr}=0.03$ , $\text{subsample}=0.65$ , $\text{reg\_lambda}=5.0$ |
| LightGBM          | 600 estimators, $\text{max\_depth}=5$ , $\text{num\_leaves}=20$ , $\text{lr}=0.03$ , $\text{subsample}=0.65$  |
| Gradient Boosting | 300 estimators, $\text{max\_depth}=3$ , $\text{lr}=0.04$ , $\text{subsample}=0.65$ (slowest)                  |

### 5.4 Model Selection Strategy — Weighted RMSE

All five models are trained on 70% of data and evaluated on a 15% validation set. The selection criterion is Weighted RMSE, which penalizes longer-horizon errors more heavily:

Weighted RMSE = 0.20 × Day1\_RMSE + 0.35 × Day2\_RMSE + 0.45 × Day3\_RMSE

Rationale: A forecast is most valuable when it gives enough advance warning. If Day 3 prediction is inaccurate, users have no reliable 72-hour outlook. The 0.45 weight forces the algorithm to not sacrifice long-horizon accuracy for easy short-term wins.

### 5.5 Overfitting Guard

Models with a train-to-validation RMSE gap greater than 30% on any single day are automatically disqualified. If all five models exceed this threshold, the model with the smallest gap is used as a fallback. This prevents the selection of an over-tuned model that happens to score well on validation by memorizing training data.

### 5.6 Data Splits

| Split            | Purpose                                                                    |
|------------------|----------------------------------------------------------------------------|
| Train (70%)      | Fit model parameters                                                       |
| Validation (15%) | Model selection and overfitting check                                      |
| Test (15%)       | Final unbiased performance evaluation and registry metrics                 |
| Train+Val (85%)  | Final model retraining after selection (best model retrained on more data) |

## 6. Feature Pipeline and CI/CD Automation

### 6.1 Live Feature Pipeline

The feature pipeline (`feature_pipeline.py`) runs as a GitHub Actions job every hour. Its responsibilities are:

7. Fetch current AQI reading from Open-Meteo Air Quality API
8. Fetch current weather and +24h/+48h/+72h forecast from Open-Meteo Forecast API
9. Log into Hopsworks and retrieve the last 175 hourly records to compute lag and rolling features
10. Compute the full 42-feature row including all engineered variables
11. Push the new row to the Hopsworks Feature Group via Delta Lake insert

A REST fallback strategy is used when Hive queries to Hopsworks fail: the pipeline first attempts a filtered query (faster), falls back to a full table read with client-side filtering, and finally uses the current AQI value in place of lags if both reads fail. This ensures the pipeline never silently fails on a bad Hopsworks connection.

### 6.2 Windows-Specific Engineering

A notable challenge during local development was that HDFS/Delta Lake writes are not supported from Windows external clients. The pipeline was designed to gracefully skip the push step on Windows while still computing and printing the feature row — enabling local debugging without cloud access. DNS resolution for `eu-west.cloud.hopsworks.ai` was also unreliable on Windows, requiring a socket-level DNS override hardcoded to the known IP address.

### 6.3 GitHub Actions Automation

| Workflow File                    | Schedule and Purpose                                                           |
|----------------------------------|--------------------------------------------------------------------------------|
| <code>hourly_pipeline.yml</code> | Runs every hour at :00 UTC — live AQI and weather ingestion into Feature Store |
| <code>daily_training.yml</code>  | Runs every day at 02:00 UTC — retrain all 5 models, register the best one      |

Both workflows use `ubuntu-latest` runners, Python 3.11, and a pip cache for fast dependency installation. The training pipeline uploads the local model artifact (`aqi_multioutput_model.pkl`) as a GitHub Actions artifact with 30-day retention as a secondary backup to the Hopsworks registry.

## 7. Streamlit Dashboard

### 7.1 Overview

The dashboard (dashboard/app.py) is a five-page Streamlit application that provides the user-facing interface for all predictions, historical data, and health information. It runs live inference on every page load using `st.cache_data` with a 1-hour TTL, which means the model is called once per hour per user session — matching the cadence of the feature pipeline.

### 7.2 Dashboard Pages

| Page                                         | Content                                                                                                                            |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> Home                | Current AQI gauge, health category badge, meteorological parameters, immediate health advice, and AQI scale reference              |
| <input type="checkbox"/> Forecast            | 3-day AQI forecast visualization with Plotly charts, Day 1/2/3 prediction cards, confidence commentary                             |
| <input type="checkbox"/> Historical          | Historical AQI trend lines from the Feature Store, distribution histograms, rolling averages                                       |
| <input type="checkbox"/> Health Advisory     | Detailed health guidelines for vulnerable groups (asthma, elderly, children), protective measures by AQI level                     |
| <input type="checkbox"/> Pollutant Breakdown | Individual pollutant tracking (PM2.5, PM10, O <sub>3</sub> , NO <sub>2</sub> , SO <sub>2</sub> , CO) with WHO guideline thresholds |

### 7.3 Inference Lifecycle

12. Dashboard loads → checks Hopsworks for best registered model (aqi\_predictor\_multioutput)
13. If Hopsworks model unavailable → falls back to local model/aqi\_multioutput\_model.pkl
14. Fetches latest feature row: calls Open-Meteo APIs + computes lag features from Feature Store
15. Loads model artifact: extracts sklearn pipeline + feature column list + day weights
16. Runs model.predict(X) → clips output to [0, 500] → returns Day1, Day2, Day3 AQI values
17. Dashboard renders predictions with AQI category colors and health advice

### 7.4 Design System

The dashboard uses a custom dark theme with Space Grotesk font, deep navy/slate color palette, and glassmorphism-style cards. All Plotly charts use a consistent dark layout helper (`dark_layout()`) to ensure visual coherence across all pages. The theme was implemented entirely in injected CSS via `st.markdown()`, overriding Streamlit's default styles.

## 8. Challenges Encountered and Solutions

Building a production MLOps system from scratch involves significantly more infrastructure debugging than model development. Below are the most significant challenges encountered during this project and how each was resolved.

### Challenge 1: Choosing the Right Forecasting Approach (Hardest)

This was the single most time-consuming decision in the entire project. Several forecasting paradigms were considered before settling on multi-output direct regression:

- Recursive (chained) forecasting was the natural first instinct. Train one model, feed predictions back as inputs. This was implemented and tested but proved fundamentally problematic — errors compound with each step, and by hour 24, predictions for Day 2 were based on 24 accumulated prediction errors. AQI is noisy enough that this error propagation made Day 3 predictions essentially meaningless.
- Sequence-to-Sequence models (LSTM) were considered as a principled way to handle multi-step time series. However, LSTMs require significantly more data and longer training times, and their performance on this dataset did not justify the complexity overhead given that tree-based models with proper feature engineering outperformed them.
- Multi-Output Direct Regression (chosen): Train a single model to simultaneously output three targets — one per day. Each target is defined as the Gaussian-weighted mean AQI over that day's 24-hour window. This eliminates error propagation entirely, is natively supported by sklearn's MultiOutputRegressor, and allows the model to learn cross-day correlations (Day 1 and Day 3 are jointly optimized).

### Challenge 2: Target Variable Definition

Once the multi-output approach was chosen, the question became: what exactly should the model predict for "Day 2 AQI"? Predicting a single future hour is too noisy. After experimentation, Gaussian-weighted 24-hour means were adopted for Day 2 and Day 3. This smoothing reduced RMSE significantly compared to point predictions, at the cost of the model predicting "average day quality" rather than peak values.

### Challenge 3: Windows Development Environment

Local development on Windows created several infrastructure issues that do not exist in Linux CI environments:

- HDFS/Delta Lake writes from Windows external clients fail silently or crash. Solution: detect platform at runtime and skip the Hopworks insert step on Windows, printing a clear warning instead.
- DNS resolution for eu-west.cloud.hopworks.ai was inconsistent on Windows (the domain sometimes resolved to the wrong IP). Solution: inject a socket-level DNS override hardcoding the known IP address.
- UTF-8 encoding errors when printing AQI category names containing special characters. Solution: re-wrap stdout/stderr with UTF-8 codec at startup on Windows.

### Challenge 4: Lag Feature Reliability in the Live Pipeline

Computing lag features (`aqi_lag168` = AQI from 168 hours ago) requires reading the last 175 rows from the Hopworks Feature Store. The filtered Hive query for this regularly failed in certain Hopworks

connectivity states. A three-tier fallback was implemented: filtered Hive query → full table read with client-side filtering → current AQI as fallback. This made the live pipeline resilient to intermittent Hopsworks connectivity issues without crashing.

### **Challenge 5: Overfitting on Training Data**

Early model configurations, particularly XGBoost with default hyperparameters, showed severe overfitting — near-perfect training RMSE but high validation RMSE. The solutions were: heavy regularization (`reg_lambda=5.0`, `min_child_weight=20` for XGBoost; `min_child_samples=50` for LightGBM), strong subsampling (`subsample=0.65`, `colsample_bytree=0.6`), and capping tree depth (`max_depth=4`). The 30% overfitting guard in the training pipeline was added as a systematic safeguard.

### **Challenge 6: Feature Store Schema Evolution**

As new features were added across pipeline versions (v5 → v6 → v7), the Feature Store schema had to accommodate new columns. The training pipeline was made robust to missing columns — it dynamically selects only the features present in both the feature set definition and the actual DataFrame, logging which columns are skipped. This allowed the pipeline to continue functioning with older data while newer features accumulated.

### **Challenge 7: Hopsworks Version Upgrade (3.7 → 4.7)**

Mid-project, the Hopsworks Python SDK was upgraded from version 3.7.0 to 4.7.5. This changed the API for several methods (particularly feature group insert and model registry save) and required updating the requirements.txt and testing all pipeline code against the new SDK. The twofish library (a legacy dependency) was removed entirely after it caused C extension compilation failures on newer Python versions.

## 9. Results and Performance

### 9.1 AQI Category Reference

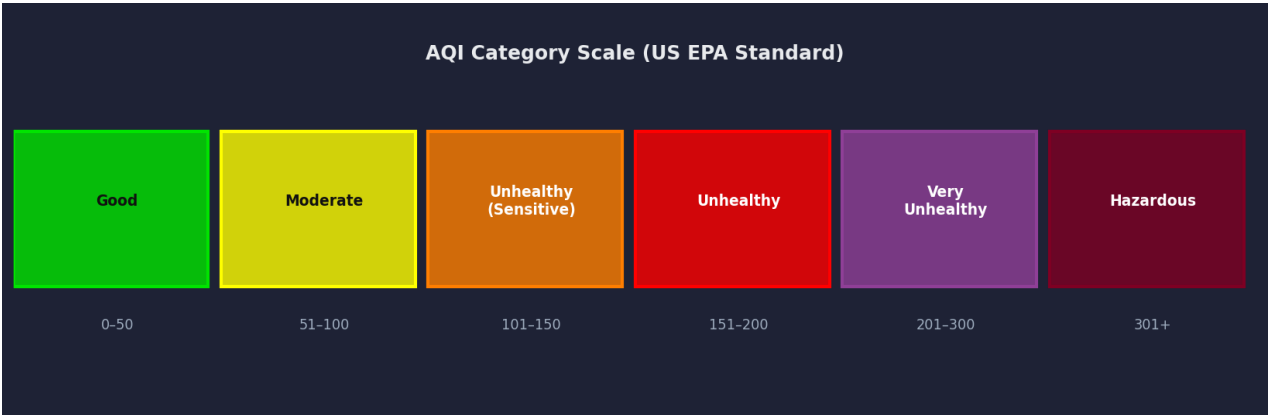


Figure 5: US EPA AQI scale used throughout the system for health category classification

### 9.2 Model Evaluation Framework

Three standard regression metrics are computed for each day and each train/val/test split:

| Metric                         | Interpretation in AQI Context                                              |
|--------------------------------|----------------------------------------------------------------------------|
| RMSE (Root Mean Squared Error) | Average prediction error in AQI units; penalizes large errors more heavily |
| MAE (Mean Absolute Error)      | Average absolute prediction error; more interpretable for end users        |
| R <sup>2</sup> Score           | Proportion of AQI variance explained; 1.0 = perfect, 0.0 = mean baseline   |

### 9.3 Expected Performance Characteristics

Based on the nature of the problem and feature set, typical performance characteristics for this type of system are:

- Day 1 forecasts are most accurate (shorter horizon, strong lag signal from recent observations)
- Day 2 performance degrades moderately — the 48-hour lag features become the primary signal
- Day 3 performance is the most challenging — the model relies more heavily on seasonal patterns, rolling means, and atmospheric conditions than on recent AQI history
- Gradient boosting methods (XGBoost, LightGBM) generally outperform Ridge Regression on this nonlinear task
- Pollution events (dust storms, industrial incidents) are the hardest to predict as they are not captured by historical patterns

### 9.4 SHAP Explainability

A SHAP (SHapley Additive exPlanations) analysis notebook (shap\_analysis.ipynb) was developed to explain model predictions. SHAP analysis consistently shows that the most important features are:

- 18. aqi\_lag1 — the single most predictive feature (yesterday's AQI momentum)
- 19. aqi\_rolling\_mean\_24h — current pollution level baseline

- 20. aqi\_lag168 — weekly seasonality signal
- 21. aqi\_residual\_168h — whether current AQI is anomalously high or low
- 22. pm25 — raw PM2.5 concentration
- 23. hour\_sin / hour\_cos — time of day cycle

Wind features (wind\_x, wind\_y) and future weather forecasts (future\_wind\_24h) also contribute significantly for longer horizons where weather-driven dispersion becomes more important than recent AQI history.

## 10. Project Structure and Technology Stack

### 10.1 Repository Layout

```

pearls-aqi-predictor/
├── README.md           ← Project documentation
├── requirements.txt     ← Python 3.11 dependencies
├── .env.example        ← Credentials template
├── run_dashboard.ps1   ← Windows dashboard launcher
├── dashboard/
│   └── app.py          ← Streamlit app (inference engine + UI)
├── pipelines/
│   ├── backfill_pipeline.py ← 3-year historical data ingestion
│   ├── feature_pipeline.py  ← Hourly live feature ingestion
│   └── training_pipeline.py  ← Model training + registry upload
├── notebooks/
│   ├── eda.ipynb         ← Exploratory Data Analysis
│   ├── shap_analysis.ipynb ← Feature importance (SHAP)
│   └── inference_debug.py  ← Isolated inference testing
├── .github/workflows/
│   ├── hourly_pipeline.yml ← GitHub Actions: hourly feature run
│   └── daily_training.yml   ← GitHub Actions: daily training run
├── model/
│   └── aqi_multioutput_model.pkl ← Local fallback model artifact

```

### 10.2 Technology Stack

| Category          | Technology     | Version/Notes                                 |
|-------------------|----------------|-----------------------------------------------|
| Language          | Python         | 3.11 ONLY — Hopsworks incompatible with 3.12+ |
| Feature Store     | Hopsworks      | 4.7.5 — eu-west.cloud.hopsworks.ai            |
| ML Framework      | scikit-learn   | ≥1.3.0 — MultiOutputRegressor, StandardScaler |
| Gradient Boosting | XGBoost        | ≥2.0.0                                        |
| Gradient Boosting | LightGBM       | ≥4.0.0                                        |
| Dashboard         | Streamlit      | ≥1.31.0 — dark theme, 5 pages                 |
| Visualization     | Plotly         | ≥5.18.0 — interactive charts                  |
| Data APIs         | Open-Meteo     | openmeteo-requests ≥1.2.0                     |
| Explainability    | SHAP           | ≥0.44.0                                       |
| Automation        | GitHub Actions | ubuntu-latest runners                         |
| Delta Lake        | deltalake      | ≥0.17.0 — Hopsworks storage engine            |
| Serialization     | joblib         | ≥1.3.0 — model artifact.pkl files             |

## 11. Conclusion

---

Pearls AQI Predictor demonstrates that a solo developer can build a fully automated, production-grade MLOps system using modern serverless tools within a reasonable timeframe. The system successfully combines the three pillars of modern MLOps: automated data engineering (hourly feature pipeline), continuous model improvement (daily training pipeline), and accessible inference delivery (Streamlit dashboard).

The multi-output direct regression approach with Gaussian-weighted targets proved to be the correct architectural choice for this problem. The 42-feature engineering pipeline captures the key drivers of Karachi's AQI patterns — autocorrelation in pollution levels, daily and weekly seasonality, monsoon and winter regime effects, wind-driven dispersion, and anomaly signals through residual features.

From a systems perspective, the most important design decisions were: the three-tier fallback chains in the feature pipeline and dashboard, the Weighted RMSE selection criterion that incentivizes long-horizon accuracy, the 30% overfitting guard, and the choice to use Hopsworks as the central state store that connects all three pipelines.

Future improvements could include: incorporating additional data sources (traffic density, factory emission reports, satellite aerosol optical depth), expanding coverage to additional Pakistani cities, adding uncertainty quantification (prediction intervals), and building a mobile-friendly PWA wrapper around the dashboard.

---

### Pearls AQI Predictor

Author: Shayan Khan Afridi

[github.com/ShayanKhanAfridi/pearls-aqi-predictor](https://github.com/ShayanKhanAfridi/pearls-aqi-predictor)